

Experiment 7:

Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook

1. Introduction to Ansible

What Is Ansible?

Ansible is an open-source IT automation and configuration management tool. It allows you to manage multiple servers and perform tasks such as:

- **Configuration Management:** Automate the configuration of servers.
- **Application Deployment:** Deploy applications consistently.
- **Orchestration:** Coordinate complex IT workflows and processes.

Key Concepts in Ansible

- **Inventory:** An inventory is a file (usually in INI or YAML format) that lists the hosts (or groups of hosts) you want to manage. It tells Ansible which machines to target.
- **Playbook:** A playbook is a YAML file that defines a set of tasks to be executed on your target hosts. It is the heart of Ansible automation. In a playbook, you specify: Hosts: The target machines (or groups) on which the tasks should run.
- **Tasks:** A list of actions (using modules) that should be executed.
- **Modules:** Reusable, standalone scripts that perform specific actions (e.g., installing packages, copying files, configuring services). Ansible comes with a large collection of built-in modules (such as apt, yum, copy, service, etc.). These modules perform specific tasks on target hosts. You can also write custom modules if needed.

Why Use Ansible?

- **Agentless:** Ansible uses SSH to communicate with target hosts, so no agent needs to be installed on them.
- **Simplicity:** Playbooks use simple YAML syntax, making them easy to write and understand.
- **Idempotence:** Ansible tasks are idempotent, meaning running the same playbook multiple times yields the same result, ensuring consistency.
- **Scalability:** Ansible can manage a small number of servers to large infrastructures with hundreds or thousands of nodes.

Installing Ansible on Ubuntu (Using Windows Subsystem for Ubuntu (WSL))

What's WSL?

WSL allows you to run a full Linux terminal inside Windows **without a VM**. It's lightweight and works perfectly for Ansible labs.

Step 1: Install WSL (Windows Subsystem for Ubuntu)

Open **PowerShell as Administrator**, and run this:

```
wsl --install
```

This will:

- Install **Ubuntu** as the default Linux distro.
- Automatically enable WSL and reboot your system.

```
wsl
```

Step 2: Open Ubuntu (WSL)

After installation and reboot:

Search for "**Ubuntu**" in the Start menu and open it and type the below codes:

```
sudo apt update
```

Now it will ask for password enter the password.

```
sudo apt install ansible -y
```

That will install Ansible successfully. Now to verify installed ansible or not, type below code:

```
ansible --version
```

Step 3: Create a Working Directory

```
mkdir ~/ansible-lab
```

```
cd ~/ansible-lab
```

Step 4: Create an Inventory File

Create a file named hosts:

```
nano hosts
```

type the below code:

```
[local]
```

```
localhost ansible_connection=local
```

Then save:

- Press CTRL + O → Enter to save
- Press CTRL + X to exit

Step 5: Test Ansible is Working

Run a simple ping module:

```
ansible -i hosts local -m ping
```

You will see below code:

```
localhost | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

Step 6: Create a Sample Playbook

Create a file named install_nginx.yml:

```
nano install_nginx.yml
```

Paste this basic playbook:

- name: Install and start NGINX on localhost
 - hosts: local
 - become: yes

tasks:

- name: Install NGINX

- apt:

- name: nginx

- state: present

- update_cache: yes

- name: Ensure NGINX is running

- service:

- name: nginx

- state: started

- enabled: yes

Step 7: Run the Playbook

ansible-playbook -i hosts install_nginx.yml --ask-become-pass

Step 8: Verify the Result

curl http://localhost